Does a Robot's Failure Monitor Still Work After a Speed-Up?

Abraham Yeung
Stanford University
ayeung16@stanford.edu

Abstract: Robots controlled by vision-language-action (VLA) models are often paired with a failure monitor: a lightweight check that raises an alarm when a task seems to be going wrong. The monitor's alarm threshold is tuned on the model as originally trained, but the deployed model is often a sped-up version, so we ask whether the monitor still works after that swap. We tune simplified versions of four common monitors on the open SmolVLA model, then run sped-up versions of it on the LIBERO-Long benchmark with the monitors left unchanged. Building each action plan in 2 refinement steps instead of 10 makes the model 2.5× faster with no observed drop in success (27% before, 32% after). Yet it disables a monitor that checks whether the robot's successive plans agree: that monitor caught about half of all failures before and catches none after, because the faster model's plans agree more closely, which pulls every score below the old threshold. Counting false alarms, the only thing the threshold's tuning guarantees, does not flag this, because false alarms drop to zero too. Rounding the model's weights to fewer bits nudges a different monitor, one that reads the model's visual and language features, toward reporting "safe". After any speed-up, a monitor should be re-tested; comparing its scores on successful runs before and after the speed-up already exposes the shift, without a single failure.

Keywords: VLA, acceleration, quantization, runtime monitoring, conformal prediction

1 Introduction

A *failure monitor* watches a robot while it works and raises an alarm when the current attempt looks likely to fail, so that a person or a fallback controller can step in. For learned robot policies, including VLAs, a monitor is usually a small scoring function that reads either the policy's internal state or the actions it plans; an alarm fires when the score crosses a threshold [1, 2, 3, 4]. A common recipe sets the threshold using only successful runs of the policy, so that at most a fixed fraction of them, say 10%, would raise an alarm [1, 3, 4, 5]. This conformal guarantee [6] limits false alarms, but only while the deployed policy behaves like the one the threshold was tuned on, and it says nothing about how many failures get caught.

The deployed policy is often not the policy as trained. To run fast enough on a robot it is sped up: its weights are stored with fewer bits (quantization), it is pruned, or it takes fewer steps to produce each action [7, 8, 9]. Papers on these speed-ups check task success; none of them evaluates the monitor. A monitor can break in two ways: its scores can stop ranking failures above successes, or they can keep ranking well while drifting past the fixed threshold. For language-model monitors, fine-tuning often degrades ranking (recoverably, by realignment) while quantization largely preserves it [10], and changing inference settings after calibration breaks conformal coverage [11]. Concurrent work finds the second kind of failure for a VLA probe under 4-bit weights, by replaying the original policy's recorded observations through the compressed model [12].

CoRL 2026 Workshop on Efficient Foundation Models for Real-Time Embodied AI, Austin TX, USA.

We test the deployed setup directly: each sped-up policy runs on the task itself, so its own actions decide what it sees next, while four unchanged monitors watch it. Code and results: https://github.com/Abraham-y/monitor-under-compression.

2 Setup

Robot policy and task. SmolVLA [13] is an open VLA with ∼450M parameters. A vision-language *backbone* (SmolVLM2) reads the camera images and the instruction. An *action expert* then plans the next 50 actions by flow matching: like a diffusion model, it turns random noise into a plan over 10 refinement steps. The robot executes only the first action and replans at the next timestep, the schedule the checkpoint ships with. We use the public LIBERO checkpoint with its own settings, run through LeRobot, on LIBERO-Long [14], a simulated suite of 10 long-horizon manipulation tasks. Each episode starts from one of 50 fixed initial states per task and fails if the task is not done within 520 timesteps. The original policy succeeds in 32% of all 500 episodes (27% on the test set), below the 71% the SmolVLA paper reports. An independent evaluation of the released model reports 35% [15], and a public reproduction that executes 10 actions per plan reports 56%.[1] Our rollout code matches LeRobot's own evaluator under this schedule (6 of 10 LIBERO-Spatial episodes each), and executing 10 actions per plan lifts test success only to 33%, so the schedule does not explain the gap.

Speed-ups. **2 steps**: 2 refinement steps instead of 10, weights untouched; the median time per policy call (batch of 10, one L4 GPU) falls from 0.81 s to 0.32 s. **Weight quantization** rounds the weights of the 521 linear layers in the backbone and action expert to 8 or 4 bits; weights in a group share one scale, and bigger groups round more coarsely. **int8** uses one scale per output channel, **int4-g32** one per 32 weights, and **int4-coarse** one per 128 weights where the layer width allows and one per row otherwise. The rounding is simulated: it reproduces low-bit accuracy, not speed.

Monitors. We fit four monitors once, on runs of the original policy, and never change them. Two read the backbone and two read the action expert.

- **VLM probe**: a linear classifier on the backbone's last-layer features that predicts whether the episode will fail.
- **Mahalanobis**: how far the backbone's features lie from those of successful training runs; it needs no failure examples (a backbone variant of VLA-FAIL's detector [2]).
- **Expert probe**: the same classifier on the action expert's internal state, which is where SAFE reads flow-matching VLAs [1]; a linear stand-in for SAFE.
- **Action consistency**: how much each new 50-action plan disagrees with the previous one where they overlap, after VLA-FAIL and ActProbe [2, 16]. Sentinel [4] instead compares distributions of many sampled plans.

A monitor's score at timestep t is the average of its per-timestep scores so far, and the alarm fires the first time that average exceeds the threshold after a 10-timestep warm-up. The threshold is set on successful runs of the original policy so that at most 10% of them would raise an alarm.

Protocol. Initial states 0–29 of every task train the monitors, 30–39 set the thresholds, and 40–49 form the test set: 100 episodes per variant, all with the same initial states and random seed. The hypotheses, outcomes (including re-tuning), splits and monitors were pre-registered before the main data collection. The int4-g32 variant and all other analyses (paired tests, best possible thresholds, score shifts, checks) were added afterwards and are exploratory.

[1] https://github.com/huggingface/lerobot/issues/3264. The released checkpoint's action expert is 0.5 times the backbone width, not the paper's 0.75. With MuJoCo ≥3.4, which we use, 152 of the 500 LIBERO-Long initial states reportedly settle differently (LeRobot issue 4390); every variant sees the same states.

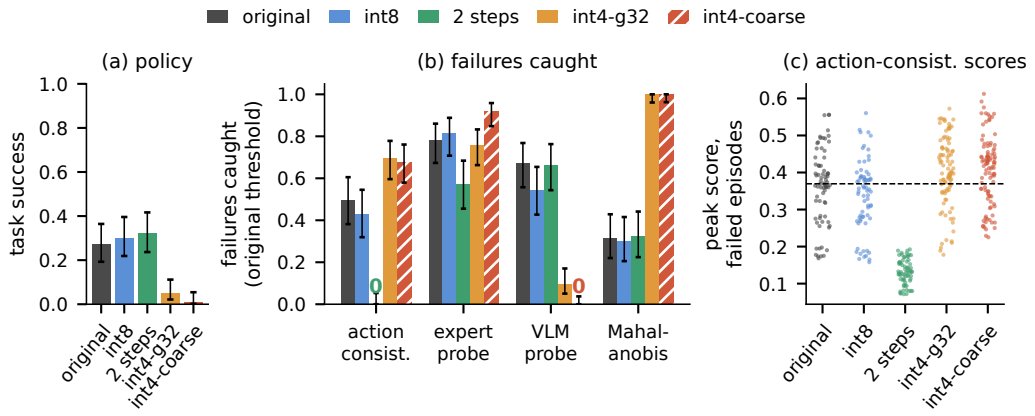


Figure 1: **(a)** Task success. **(b)** Fraction of failures each monitor catches with the original threshold. Both int4 variants break the policy, so their high bars do not mean good monitoring. **(c)** Each dot is a failed episode's peak action-consistency score; dots below the dashed original threshold are missed. Error bars: Wilson 95% intervals over the 100 test episodes (a) or failed ones (b), not clustered by task.

Table 1: Original → 2-step policy, test set. *Caught*, *false alarms*: with the original threshold. *Best possible*: most failures any threshold catches at ≤10% false alarms, chosen with hindsight (caught can exceed it when the original threshold gives over 10% false alarms). *AUROC*: chance a random failure outscores a random success. *Re-tuned*: threshold reset on the 2-step policy's successful runs.

monitor	failures caught	false alarms	best possible	AUROC	re-tuned: caught / false al.
action consistency	.49→**.00**	.15→.00	.44→.34	.77→.72	.26 / .03
expert probe	.78→.57	.11→.00	.77→.71	.95→.94	.66 / .06
VLM probe	.67→.66	.04→.00	.81→.87	.95→.96	.60 / .00
Mahalanobis	.32→.32	.07→.06	.36→.38	.85→.86	.29 / .06

3 Results

Two refinement steps disable the action-consistency monitor. The 2-step policy succeeds in 32% of test episodes, against 27%: no observed drop (10 initial states gained, 5 lost; 95% interval over tasks: -1 to $+12$ points), not proof of parity.

- **Action consistency** caught 49% of the original's failures (36 of 73) and catches none of the 2-step policy's 68 (Fig. 1b,c). On initial states where both fail, it misses 30 failures it used to catch and gains none (p=2×10^{-9}, paired McNemar; 95% interval over tasks and thresholds: -0.92 to -0.25). It stays at zero with a stricter 5% false-alarm target.
- **The expert probe** drops from 78% to 57% (14 lost, 0 gained), and its median alarm moves from timestep 10, the earliest allowed, to 124. But the interval over tasks (-0.41 to $+0.08$) includes zero and the drop vanishes at a 5% target, so we treat it as suggestive.
- **The backbone monitors** barely move: VLM probe 66% against 67%, Mahalanobis 32% against 32%. Fewer refinement steps change only the action expert; on the first timestep, where both versions see the same image, the backbone's features are bit-for-bit identical.[2]

Most of the loss is the inherited threshold; some is ranking. The score shrinks by a roughly constant factor: on the same initial state, successive plans' disagreement over the first 30 timesteps is 0.42 times its original value (middle half of episodes: 0.40–0.44), and 0.43 for runs that succeed

[2]On 6 of the 10 tasks, and on 19 of 20 episodes of a seventh. On the other 3, the original's first-timestep features differ slightly, likely from GPU kernel autotuning. Restricted to the 6 matched tasks, the detection results hold except int8's.

in both. This fits few-step Euler sampling, which keeps the mean plan but shrinks its spread, to 0.43–0.51 of the 10-step spread for π_0 and $\pi_{0.5}$ [17].

- **Ranking weakens but survives** (Table 1): AUROC goes from 0.77 to 0.72, and the best threshold chosen with hindsight catches 34% of failures, against 44% before. Ranking thus costs about 10 points; the old threshold, now above every score, costs the other 34. Re-tuning it on the 2-step policy's successes recovers 26%.
- **The failure looks like an improvement.** False alarms fall to zero (from 15% for action consistency, 11% for the expert probe). A team that checks the false-alarm rate, the one thing the threshold guarantees, would see the monitor getting better.

Set on successes, as is common, the threshold turns the shift into fewer false alarms that hide the loss, even for a monitor with no trained weights. Concurrent work finds ranking intact but the threshold wrong for quantized probes [12].

Weight quantization nudges the VLM probe toward "safe". On the first timestep, where every variant sees the same image, the VLM probe's score moves toward "safe" by 0.29 standard deviations of the original's first-timestep scores under int8 (in all 120 matched episodes), 2.3 under int4-g32 and 4.6 under int4-coarse. The share of failures it catches falls from 67% to 54% (int8; 13 lost, 3 gained, p=0.02, but not significant on the matched tasks), 9% and 0%. Under int8, task success and false alarms show no detectable change. Both int4 settings break the policy itself (5% and 1% success): under int4-coarse the probe stays silent through all 99 failures, while Mahalanobis alarms on every int4 episode, successes included, so it detects the compression rather than the failures.

Checks. *Episode length:* failed episodes run to the time limit, giving monitors more chances to alarm [12]. Counting only alarms before timestep 200 lowers original detection (VLM probe 0.67→0.48, action consistency 0.49→0.27), but both zeros stay at zero. *Task identity:* two tasks never succeed, so a probe could score well by recognizing them; but the original within-task AUROC is still 0.96 (VLM probe) and 0.98 (expert probe), and dropping those tasks changes no conclusion. *Pre-registration:* both primary hypotheses concerned int4 and were uninformative (one int4-coarse test success). Of the secondary ones, 2 steps moved the expert probe's false-alarm rate more than int8 did (held), and int8 moved every false-alarm rate by at most 0.05 but one AUROC by 0.08 (failed). We did not predict the detection collapse.

4 Related work and discussion

Related work. Monitors for VLAs and other learned policies are evaluated with the policy fixed [1, 2, 3, 4]; even then, a SAFE-style monitor on held-out tasks can far exceed its false-alarm target [18]. SAFECAST [19] studies thresholds under input changes. Monitor staleness under model updates has been studied for language models [10, 11] and, concurrently, for a quantized VLA replayed on recorded inputs, repaired by realigning its activations [12]. Speed-up studies measure success, speed and motion quality [7, 8, 20]. CARE could not certify fewer flow steps for $\pi_{0.5}$ on LIBERO-Long [9]; for SmolVLA, fewer steps keep or raise success [15]; we lose only monitoring.

Takeaways. Treat every speed-up as invalidating a monitor's threshold: neither unchanged success nor fewer false alarms shows that failures are still caught. Compare its scores on successful runs before and after: here the shrink (median 0.27 vs. 0.11) shows without a single failure. Then re-test detection on the sped-up policy's real failures. Expect a monitor to break where its input changes.

Limitations. One policy, task suite, replanning schedule and seed, with 100 test episodes per variant; low baseline success; simulated quantization. Our monitors simplify SAFE, VLA-FAIL and Sentinel rather than reproduce them (SAFE uses a time-varying threshold). int4-coarse was an unintended setting that code review caught.

Acknowledgments

We thank Modal for compute.

References

[1] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, et al. SAFE: Multitask failure detection for vision-language-action models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[2] F. Seligmann, E. Gospodinov, E. U. Dincer, and G. Neumann. VLA-FAIL: Efficient task failure detection for finetuned vision-language-action models. In *Conference on Robot Learning*, 2026.

[3] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig. Failure prediction at runtime for generative robot policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[4] C. Agia, R. Sinha, J. Yang, Z.-a. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning*, 2024.

[5] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, et al. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems (RSS)*, 2025.

[6] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[7] J. Xu, Q. Jin, M. Chen, S. Wang, Y. Sui, and Y. Shi. VLAQuantBench: Closed-loop evaluation of post-training quantization for vision-language-action models. *arXiv preprint arXiv:2609.25376*, 2026.

[8] J. Zhang, Y. Hsieh, Z. Wan, H. Lin, X. Wang, Z. Wang, Y. Lei, and M. Zhang. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[9] R. Liu, T. Zheng, J. Gu, and Z. Wang. CARE: Certifying acceleration for vision-language-action inference. *arXiv preprint arXiv:2610.08917*, 2026.

[10] E. Duan. Do activation monitors survive model updates? Benchmarking, predicting, and repairing activation-monitor staleness. *arXiv preprint arXiv:2606.15980*, 2026.

[11] A. Doula and M. Mühlhäuser. Auditing conformal prediction for language models under inference-time model shift. In *Advances in Neural Information Processing Systems (NeurIPS), Evaluations and Datasets Track*, 2026.

[12] A. Bysani. Rank-preserving, threshold-breaking: Post-training silently decalibrates inherited safety monitors in VLA policies. In *NeurIPS 2026 Workshop on Post-Training Adaptation of Robot Foundation Models (RoboPAD)*, 2026.

[13] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[14] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2023.

[15] S. Sujit, L. Nunziante, R. F. J. Dossa, and K. Arulkumaran. When one step is enough: An empirical study of Euler integration in flow-matching policies. *arXiv preprint arXiv:2610.30735*, 2026.

[16] B. Huang, X. Li, X. Wang, et al. ActProbe: Action-space probe for early failure detection of generative robot policies. *arXiv preprint arXiv:2606.08508*, 2026.

[17] J. Yang, Y. Xu, Y. Chen, Q. Ma, P. Li, et al. Coupling execution horizon and sampling depth in flow-matching VLA policies. *arXiv preprint arXiv:2610.32102*, 2026.

[18] D. Ahmedt-Aristizabal, A. Khamis, M. A. Armin, and L. Petersson. RESCUE: Is this alarm worth acting on? Counterfactual evaluation of VLA runtime monitors. *arXiv preprint arXiv:2610.13516*, 2026.

[19] H. Rajaprakash, A. Prajapati, R. Xue, A. Anwar, and J. Thomason. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration. *arXiv preprint arXiv:2608.04246*, 2026.

[20] Z. Li, H. Yang, Z. Chen, Y. Chen, et al. From inference efficiency to embodied efficiency: Revisiting efficiency metrics for vision-language-action models. *arXiv preprint arXiv:2603.19131*, 2026.